



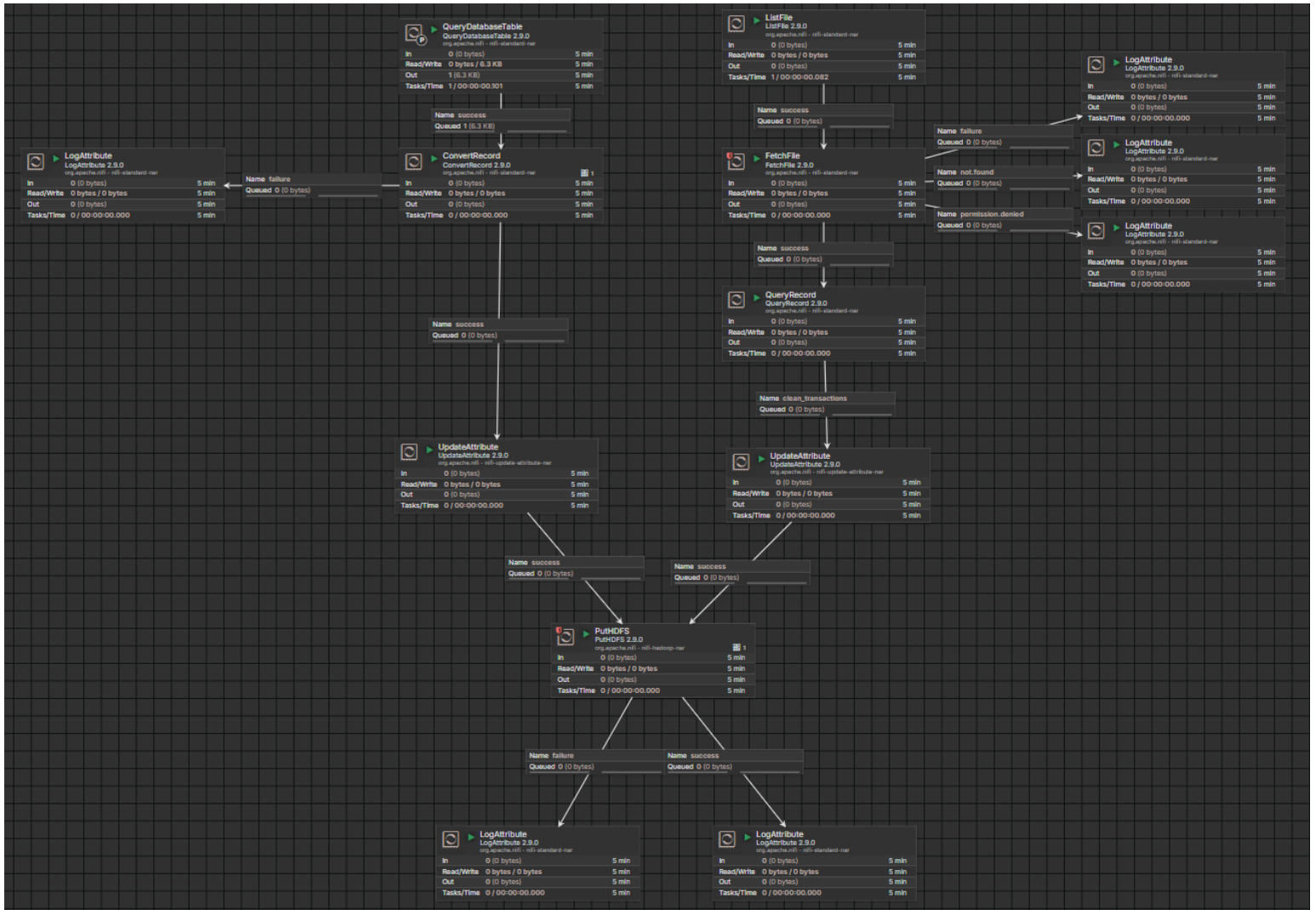
APACHE NIFI DATA PIPELINE PROJECT

NIFI Assignment



MAY 10, 2026

FADI AL-ARIQI





Simulation Script:

```
import pandas as pd
import numpy as np
import os
import time
from datetime import datetime
import random

OUTPUT_DIR = r"./my_data/input"
if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)

def generate_messy_data(num_rows=10):
    data = {
        'transaction_id': [random.randint(1000, 2000) for _ in range(num_rows)],
        'customer_id': [random.randint(1, 100) for _ in range(num_rows)],
        'amount': [round(random.uniform(10.5, 500.0), 2) for _ in range(num_rows)],
        'status': random.choices(['Success', 'Failed', 'Pending', None], k=num_rows),
        'timestamp': [datetime.now().strftime("%Y-%m-%d %H:%M:%S") for _ in
range(num_rows)]
    }

    df = pd.DataFrame(data)

    if random.random() > 0.7:
        df = pd.concat([df, df.iloc[[0]]], ignore_index=True)

    return df
```

```
def start_simulation(max_files=20):
    print(f"Starting data simulation in: {OUTPUT_DIR}")
    file_count = 0

    try:
        while file_count < max_files:
            timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")
            file_name = f"transaction_{timestamp_str}.csv"
            file_path = os.path.join(OUTPUT_DIR, file_name)

            df = generate_messy_data(random.randint(5, 15))
            df.to_csv(file_path, index=False)

            file_count += 1
            print(f"[{file_count}/{max_files}] Created: {file_name}")

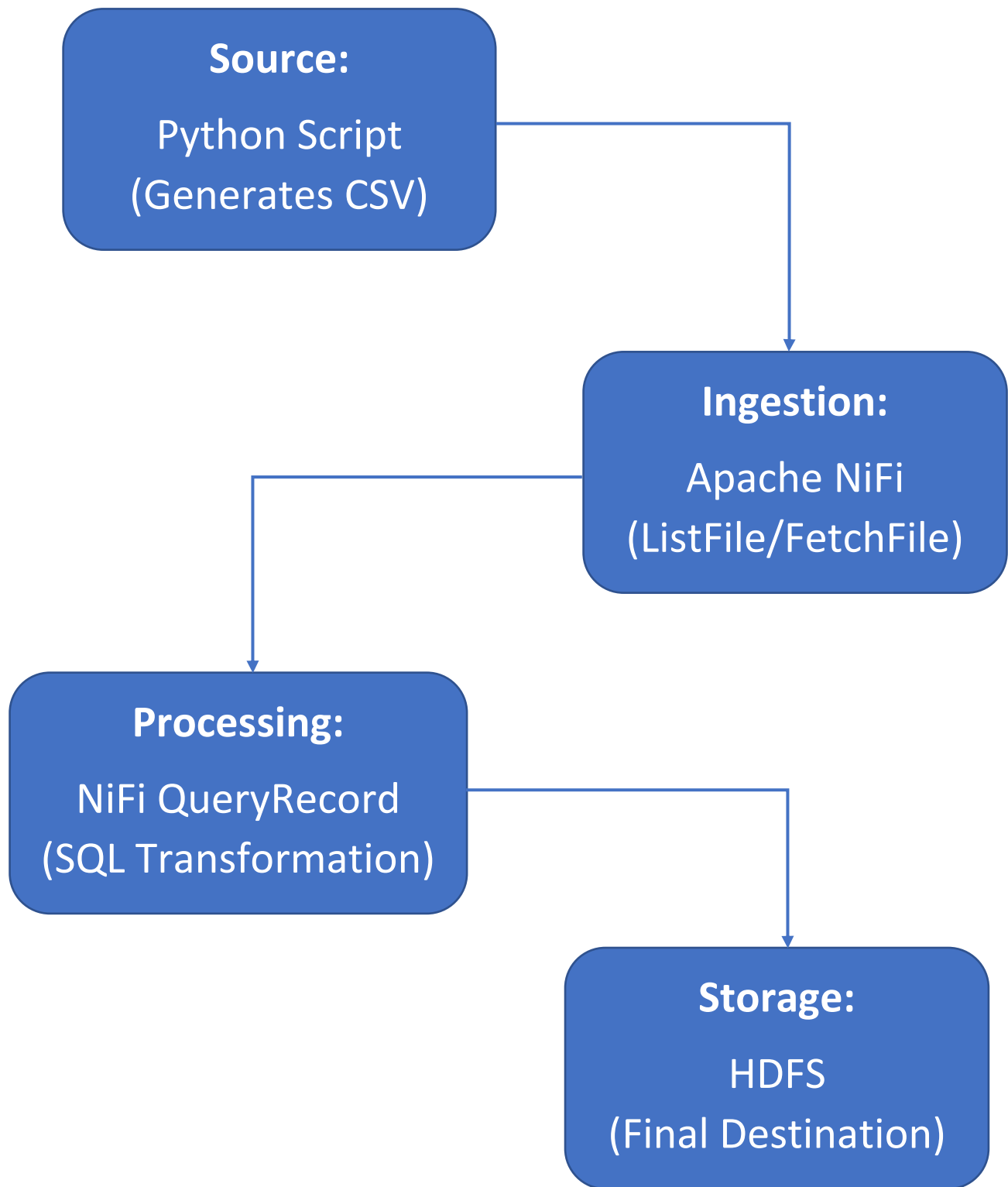
            time.sleep(5)

    except KeyboardInterrupt:
        print("\nSimulation stopped by user.")

if __name__ == "__main__":
    start_simulation(max_files=20)
```



Architecture Diagram





Design Decisions & Technical Documentation

Project: Real-Time Data Ingestion Pipeline via Apache NiFi

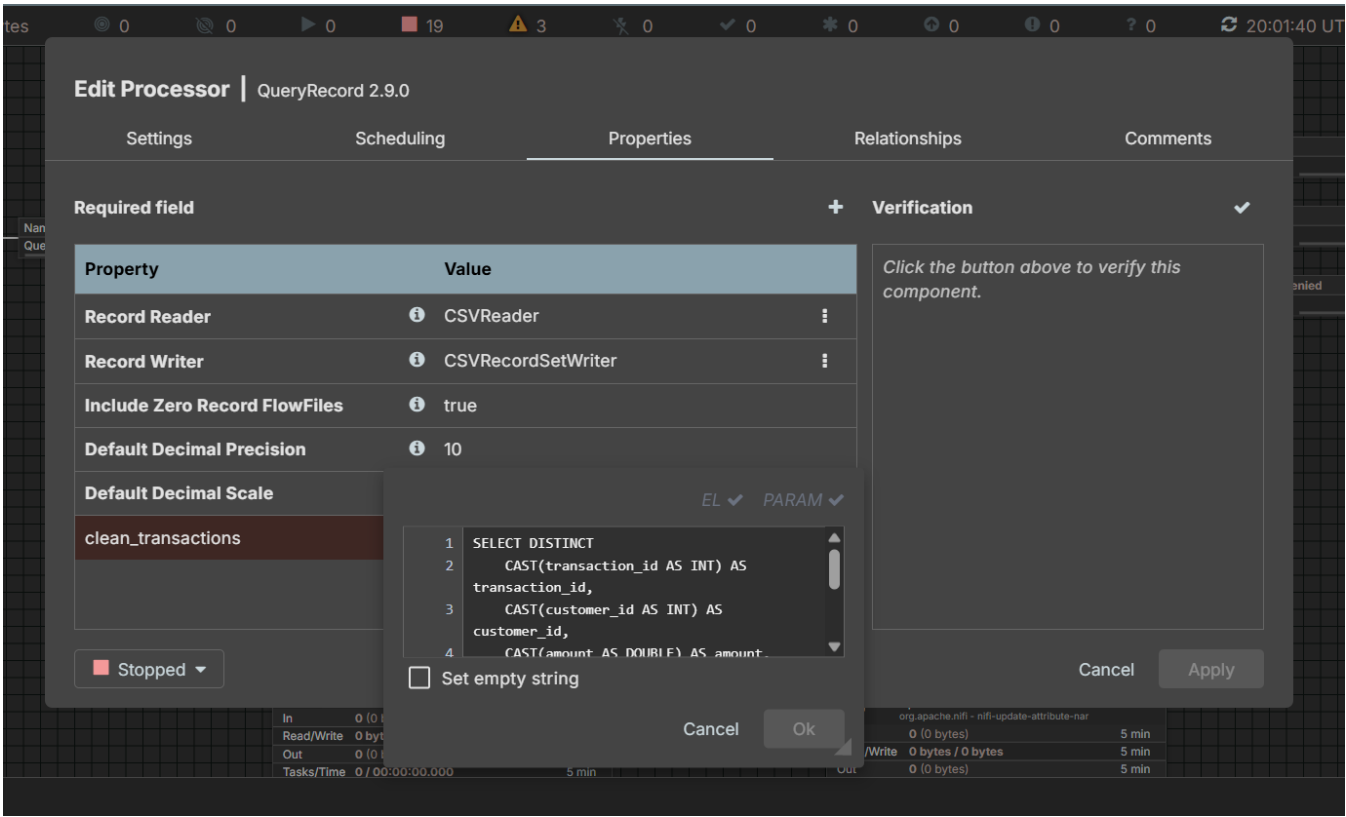
1. Architectural Overview

The solution is designed as a multi-source data pipeline that bridges local file systems and relational databases into a centralized Hadoop Distributed File System (HDFS).

2. Key Design Decisions

A. Unified Transformation via QueryRecord

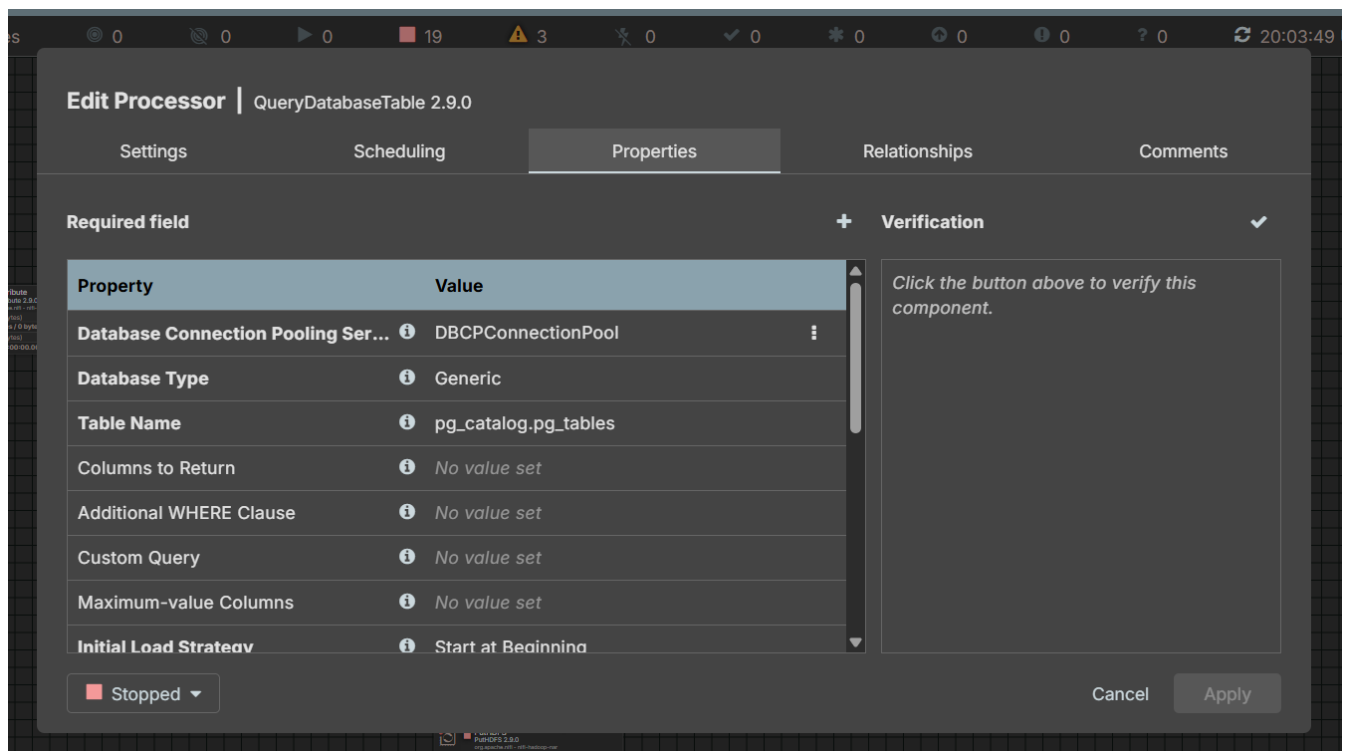
Instead of using a chain of specialized processors (like DeduplicateRecord, ValidateRecord, and UpdateRecord), I implemented a single QueryRecord processor using Apache Calcite SQL.



Reasoning: This decision minimizes "Disk I/O" and reduces flow complexity. By performing filtering (removing NULLs), deduplication (SELECT DISTINCT), and type casting (CAST to INT/DOUBLE) in one in-memory step, the pipeline achieves higher throughput and is significantly easier to maintain.

B. Incremental Database Ingestion

For the PostgreSQL (Hive Metastore) ingestion, I utilized the QueryDatabaseTable processor instead of standard SQL execution.



Reasoning: This allows for Incremental Extraction. By configuring a Maximum-value Column (like DB_ID), NiFi tracks the state and only fetches new records. This prevents data redundancy and protects the source database from unnecessary load.

C. Resilience via Back Pressure & Yielding

I applied NiFi Best Practices by configuring Back Pressure Object Thresholds (set to 2000) on the connections leading to HDFS.

Reasoning: This acts as a safety valve. If the HDFS cluster becomes slow or unreachable, NiFi will automatically pause the upstream processors, preventing memory exhaustion and ensuring no data loss occurs.

3. Technical Challenges & Resolutions

Challenge	Solution
Schema Mismatch	Encountered a "Column not found" error due to different column names in the Python script. Resolved by synchronizing the SQL schema in QueryRecord with the CSV header.
Reserved Keywords	The column timestamp caused a SQL parsing error. Resolved by using SQL Quoting Identifiers ("timestamp") to allow the engine to treat it as a literal column name.
Incremental State	Ensuring the DB didn't reload the same data. Resolved by using NiFi's State Management within the QueryDatabaseTable processor.

4. Pipeline Best Practices Demonstrated

- **Retry & Penalization:** Configured for database connection failures to ensure transient network issues don't stop the entire flow.
- **Format Conversion:** Seamlessly converted messy CSV and Avro (DB) data into structured JSON for downstream analytics.
- **Separation of Concerns:** Used different Controller Services (CSVReader, JSONWriter) to handle data formats independently from the processing logic.



Evidence of Data Flow (Execution Screenshots)

Step 1: Data Generation (Python Simulation)

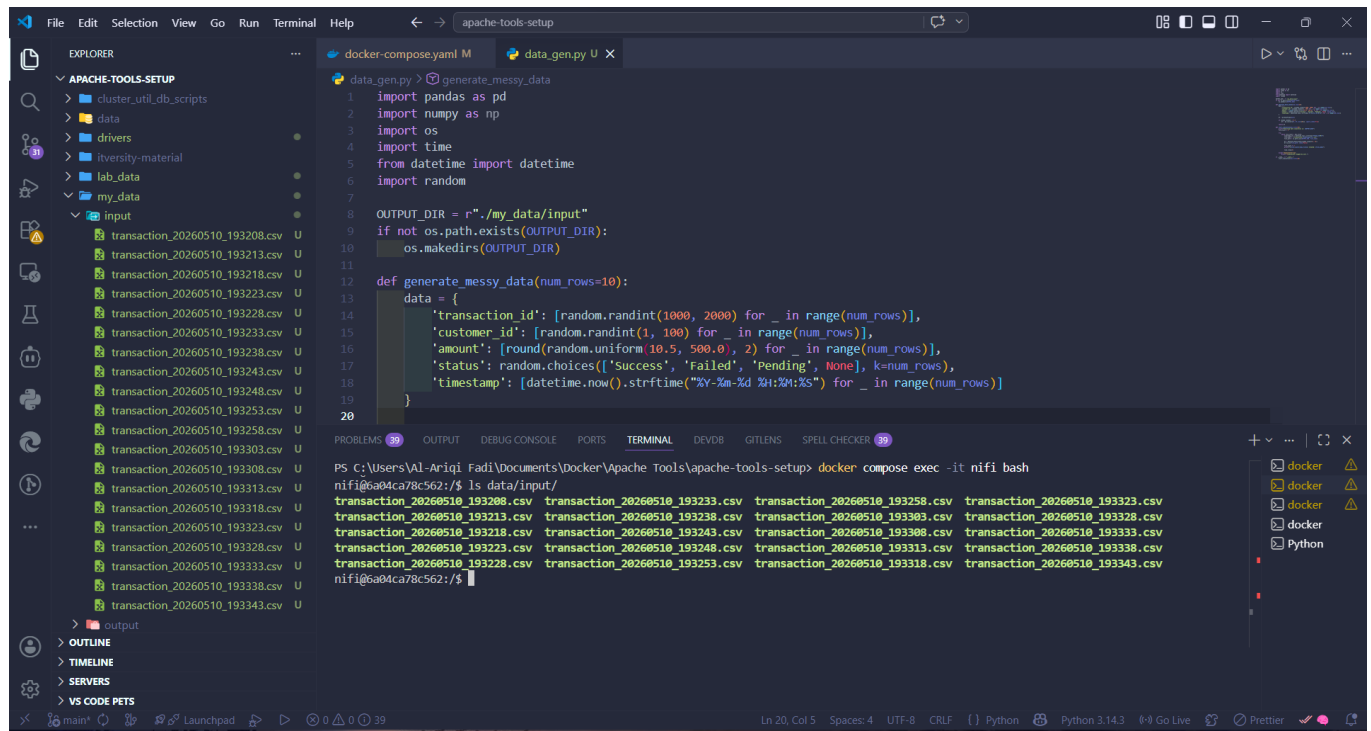
```
File Edit Selection View Go Run Terminal Help
apache-tools-setup
EXPLORER
APACHE-TOOLS-SETUP
  cluster_util_db_scripts
  data
  drivers
  itersity-material
  lab_data
  my_data
    input
      transaction_20260510_193208.csv
      transaction_20260510_193213.csv
      transaction_20260510_193218.csv
      transaction_20260510_193223.csv
    output
  nifi_hadoop_configs
  nifi-libs
  .env
  .env.example
  .gitignore
  data_gen.py
  docker-compose.yaml
  LICENSE
  README.md
  RUNNING.md
  setup_hive.sh
  ...
OUTLINE
TIMELINE
SERVERS
VS CODE PETS
data_gen.py
1 import pandas as pd
2 import numpy as np
3 import os
4 import time
5 from datetime import datetime
6 import random
7
8 OUTPUT_DIR = r"./my_data/input"
9 if not os.path.exists(OUTPUT_DIR):
10     os.makedirs(OUTPUT_DIR)
11
12 def generate_messy_data(num_rows=10):
13     data = {
14         'transaction_id': [random.randint(1000, 2000) for _ in range(num_rows)],
15         'customer_id': [random.randint(1, 100) for _ in range(num_rows)],
16         'amount': [round(random.uniform(10.5, 500.0), 2) for _ in range(num_rows)],
17         'status': random.choices(['Success', 'Failed', 'Pending', None], k=num_rows),
18         'timestamp': [datetime.now().strftime("%Y-%m-%d %H:%M:%S") for _ in range(num_rows)]
19     }
20
21     df = pd.DataFrame(data)
22
23     if random.random() > 0.7:
24         df = pd.concat([df, df.iloc[[0]]], ignore_index=True)
25
26     df.to_csv(f"{OUTPUT_DIR}/transaction_{datetime.now().strftime('%Y%m%d%H%M%S')}.csv", index=False)
27
28 if __name__ == '__main__':
29     generate_messy_data(10)
```

```
PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\apache-tools-setup> python data_gen.py
Starting data simulation in: ./my_data/input
[1/20] Created: transaction_20260510_193208.csv
[2/20] Created: transaction_20260510_193213.csv
[3/20] Created: transaction_20260510_193218.csv
[4/20] Created: transaction_20260510_193223.csv
```

Description:

I developed a custom Python script to simulate real-time messy data generation. The script creates CSV files containing transaction details with intentional anomalies (missing values and duplicates). These files are automatically saved into the local directory `./my_data/input`.

Step 2: Volume Mapping & Ingestion (Docker to NiFi)



The screenshot shows a VS Code editor with two open files: `docker-compose.yaml` and `data_gen.py`. The `data_gen.py` file contains a Python script that generates messy data and writes it to a directory. The terminal window shows the command `docker compose exec -it nifi bash` being executed, and the output of the `ls` command in the `nifi` container, which lists the files in the `data/input` directory.

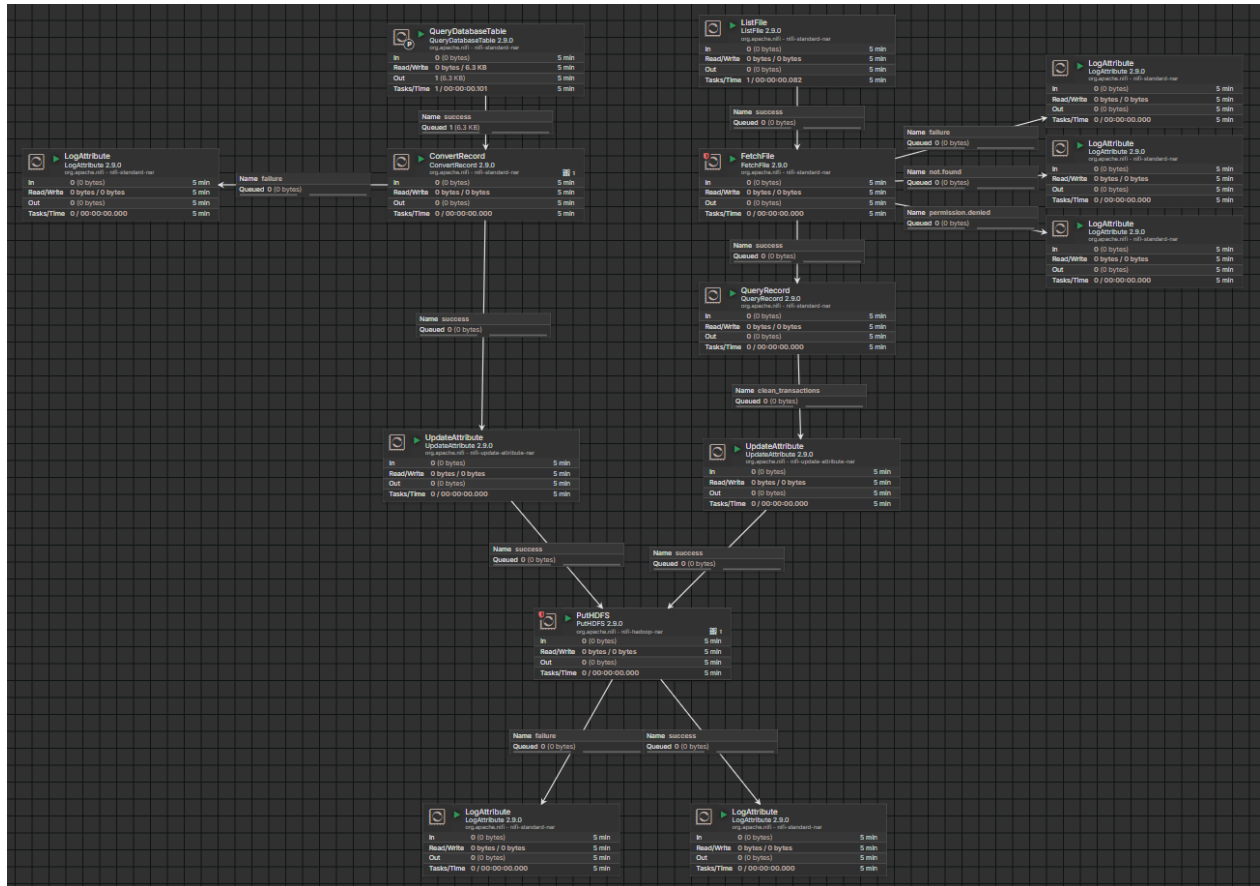
```
data_gen.py > generate_messy_data
1 import pandas as pd
2 import numpy as np
3 import os
4 import time
5 from datetime import datetime
6 import random
7
8 OUTPUT_DIR = r"./my_data/input"
9 if not os.path.exists(OUTPUT_DIR):
10     os.makedirs(OUTPUT_DIR)
11
12 def generate_messy_data(num_rows=10):
13     data = {
14         'transaction_id': [random.randint(1000, 2000) for _ in range(num_rows)],
15         'customer_id': [random.randint(1, 100) for _ in range(num_rows)],
16         'amount': [round(random.uniform(10.5, 500.0), 2) for _ in range(num_rows)],
17         'status': random.choices(['Success', 'Failed', 'Pending', None], k=num_rows),
18         'timestamp': [datetime.now().strftime("%Y-%m-%d %H:%M:%S") for _ in range(num_rows)]
19     }
20
```

```
PS C:\Users\Al-Ariqi\Fadi\documents\Docker\Apache Tools\apache-tools-setup> docker compose exec -it nifi bash
nifi@6a9dca78c562:/# ls data/input/
transaction_20260510_193208.csv transaction_20260510_193213.csv transaction_20260510_193218.csv transaction_20260510_193223.csv transaction_20260510_193228.csv transaction_20260510_193233.csv transaction_20260510_193238.csv transaction_20260510_193243.csv transaction_20260510_193248.csv transaction_20260510_193253.csv transaction_20260510_193258.csv transaction_20260510_193303.csv transaction_20260510_193308.csv transaction_20260510_193313.csv transaction_20260510_193318.csv transaction_20260510_193323.csv transaction_20260510_193328.csv transaction_20260510_193333.csv transaction_20260510_193338.csv transaction_20260510_193343.csv
```

Description:

Using Docker Volume Mapping (`./my_data/input:/data/input`), the generated files are instantly synchronized between the host machine and the NiFi container. I utilized the ListFile and FetchFile processors to monitor this directory and ingest new data into the pipeline as soon as it appears.

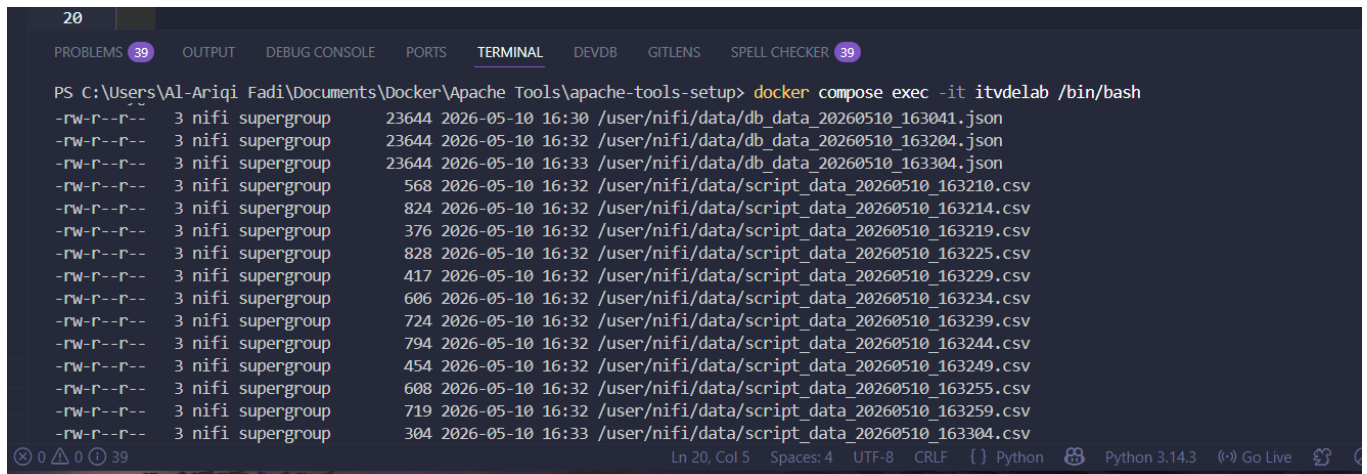
Step 3: Data Transformation (NiFi Processing)



Description:

This stage shows the core processing logic. The QueryRecord processor filters out records with null values and casts data types to their correct formats. You can observe the FlowFiles moving through the queues, indicating successful real-time processing and schema validation.

Step 4: Final Storage (HDFS Verification)



```
PS C:\Users\Al-Ariqi Fadi\Documents\Docke\Apache Tools\apache-tools-setup> docker compose exec -it itvdelab /bin/bash
-rw-r--r-- 3 nifi supergroup 23644 2026-05-10 16:30 /user/nifi/data/db_data_20260510_163041.json
-rw-r--r-- 3 nifi supergroup 23644 2026-05-10 16:32 /user/nifi/data/db_data_20260510_163204.json
-rw-r--r-- 3 nifi supergroup 23644 2026-05-10 16:33 /user/nifi/data/db_data_20260510_163304.json
-rw-r--r-- 3 nifi supergroup 568 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163210.csv
-rw-r--r-- 3 nifi supergroup 824 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163214.csv
-rw-r--r-- 3 nifi supergroup 376 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163219.csv
-rw-r--r-- 3 nifi supergroup 828 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163225.csv
-rw-r--r-- 3 nifi supergroup 417 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163229.csv
-rw-r--r-- 3 nifi supergroup 606 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163234.csv
-rw-r--r-- 3 nifi supergroup 724 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163239.csv
-rw-r--r-- 3 nifi supergroup 794 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163244.csv
-rw-r--r-- 3 nifi supergroup 454 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163249.csv
-rw-r--r-- 3 nifi supergroup 608 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163255.csv
-rw-r--r-- 3 nifi supergroup 719 2026-05-10 16:32 /user/nifi/data/script_data_20260510_163259.csv
-rw-r--r-- 3 nifi supergroup 304 2026-05-10 16:33 /user/nifi/data/script_data_20260510_163304.csv
```

Description:

The final output is routed to the PutHDFS processor. As shown in the HDFS User Interface (Namenode), the processed files are stored in the /user/nifi/data directory. The data is now converted to a clean JSON format, partitioned by arrival time, and ready for downstream analytical workloads or Hive queries.



Sample Data: Before vs. After Transformation

Scenario: Automated Deduplication Logic

File Edit Selection View Go Run Terminal Help

apache-tools-setup

docker-compose.yaml M data_gen.py U transaction_20260510_193218.csv U sample.csv U

my_data > input > transaction_20260510_193218.csv

Transaction_id	Customer_id	Amount	Status	Timestamp
1767	36	113.13	Success	2026-05-10 19:3
1549	47	291.02	Success	2026-05-10 19:3
1353	55	381.22	Success	2026-05-10 19:3
1540	49	251.53	Pending	2026-05-10 19:3
1645	45	340.61	Failed	2026-05-10 19:3
1258	25	281.07	Failed	2026-05-10 19:3
1767	36	113.13	Success	2026-05-10 19:3
*				

sample.csv

Transaction_id	Customer_id	Amount	Status	Timestamp
1549	47	291.02	Success	2026-05-10 19:3
1258	25	281.07	Failed	2026-05-10 19:3
1353	55	381.22	Success	2026-05-10 19:3
1645	45	340.61	Failed	2026-05-10 19:3
1767	36	113.13	Success	2026-05-10 19:3
1540	49	251.53	Pending	2026-05-10 19:3
*				

powered by: Wijmo FlexGrid

powered by: Wijmo FlexGrid

PROBLEMS 39 OUTPUT DEBUG CONSOLE PORTS TERMINAL DEVD8 GITLENS SPELL CHECKER 39

PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\apache-tools-setup> docker compose exec -it itvdelab /bin/bash

itversity@itvdelab:/\$ hdfs dfs -cat /user/nifi/data/script_data_20260510_163219.csv

transaction_id,customer_id,amount,status,timestamp

1549,47,291.0199890136719,Success,2026-05-10 19:32:18

1258,25,281.07000732421875,Failed,2026-05-10 19:32:18

1353,55,381.2200012207031,Success,2026-05-10 19:32:18

1645,45,340.6099853515625,Failed,2026-05-10 19:32:18

1767,36,113.12999725341797,Success,2026-05-10 19:32:18

1540,49,251.52999877929688,Pending,2026-05-10 19:32:18

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

itversity@itvdelab:/\$

Technical Explanation:

The effectiveness of the QueryRecord processor is clearly demonstrated in the provided screenshot.

Before Transformation: The raw CSV input contained redundant data, specifically the record:

1767, 36, 113.12999725341797, Success, 2026-05-10 19:32:18 which appeared multiple times.

After Transformation: By applying the SELECT DISTINCT SQL logic, Apache NiFi identified the duplicate entry in real-time. The redundant row was removed, and only a single, unique instance of the transaction was passed through to the final storage in HDFS.

Conclusion:

This ensures Data Integrity and prevents "Data Inflation" in the Hadoop ecosystem, which is a critical requirement for accurate financial reporting and analytics.